Hoja de Trabajo #1

Load Balancer

PONDERACIÓN: 3

Horas Aproximadas: 3

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas

Introducción

Objetivo: Aprender a implementar y desplegar aplicaciones web APIs en múltiples instancias EC2 y configurar un Load Balancer que distribuya el tráfico entre ellas de forma eficiente y confiable.

En esta práctica se desplegarán dos APIs web en dos máquinas virtuales EC2 independientes para luego configurar un Load Balancer para distribuir el tráfico entrante entre ambas instancias.

Este ejercicio introduce conceptos clave de arquitectura cloud como:

- Alta disponibilidad
- Balanceo de carga y tolerancia a fallos

Relación con el Curso

El balanceo de carga es un componente esencial en aplicaciones modernas desplegadas en la nube ya que permite mejorar el rendimiento, asegurar la disponibilidad continua del servicio y manejar aumentos en la demanda sin interrupciones.

Esta actividad pone en práctica estos principios, brindando al estudiante una visión realista de cómo operan las aplicaciones en entornos cloud productivos.

Competencia(s)

- Explica el concepto de Cloud Computing, sus modelos de servicio y las ventajas y limitaciones de distintos proveedores Cloud mediante ejemplos cotidianos, casos de uso empresarial y comparativas para el desarrollo de infraestructuras escalables y rentables.
- Implementa servicios básicos de Cloud utilizando Amazon EC2, Amazon RDS, Amazon S3, Azure Virtual Machines, Azure SQL, Azure Blob Storage, Azure App Services y Azure Logic Apps para desplegar aplicaciones básicas, considerando la configuración de redes, seguridad y escalabilidad.
- Aplica principios básicos de ingeniería a través de conceptos de Cloud Computing para el desarrollo de aplicaciones básicas

Contenido

Parte 1: Creación de las APIs

Indicaciones:

1. Crear 2 web APIs empleando el lenguaje y framework de su elección [preferentemente javascript y python]
2. Ambas APIs deberán poseer 2 endpoints:
    a. **Endpoint 1 ("/check"):** Endpoint utilizado para la verificación de estado, debe retornar un código 200 [OK].
    b. **Endpoint 2 ("/"):** Endpoint utilizado para obtener el objeto JSON respectivo para el API.

Consideraciones:

1. Cada API debe desarrollarse en un lenguaje diferente obligatoriamente.
2. El Endpoint 2 ("/") deberá retornar un objeto JSON para cada API. Este deberá poseer la siguiente estructura:

API #1

```
{
    "Instancia": "Instancia #1 - API #1",
    "Curso": "Seminario de Sistemas 1",
    "Estudiante": "Estudiante - <#Carnet>"
}
```

API #2

```
{
    "Instancia": "Instancia #2 - API #2",
    "Curso": "Seminario de Sistemas 1",
    "Estudiante": "Estudiante - <#Carnet>"
}
```

Parte 2: Creación de instancias EC2, despliegue de APIs y Creación de ELB

Indicaciones:

1. Crear 2 instancias de EC2, estas deberán llamarse "Instancia-1" e "Instancia-2".
2. A cada instancia de EC2 se le deberá cargar un API de la siguiente forma:
    a. A la "Instancia-1" se le deberá desplegar el "API #1".
    b. A la "Instancia-2" se le deberá desplegar el "API #2".
3. Crear el balanceador de carga , configurar correctamente el agente de escucha, la comprobación de estado y agregar ambas instancias al listado de instancias destino.

Consideraciones:

1. Para realizar el despliegue de las APIs, se puede copiar el proyecto por medio de la herramienta Termius utilizando la opcion "SFTP", clonando un repositorio público de github, creando el proyecto directamente en la instancia y modificándolo según se indica o por el medio que mejor le convenga al estudiante.
2. El nombre del balanceador de carga debe seguir el siguiente formato: *elb-semi1-ht1-#carné*

Recursos Adicionales

- **AWS Docuementation:** https://docs.aws.amazon.com/elasticloadbalancing/latest/application/create-application-load-balancer.html

Forma de entrega

Se debe entregar el URL de una grabación **no mayor de 10 minutos** en UEDI, dicha grabación debe contener lo siguiente:

1. Configuración de ambas instancias en EC2, mostrando la dirección IPv4 de cada una.
2. La prueba donde se verifique que consumiendo ambas instancias directamente de sus IPv4 se obtiene una respuesta con el objeto JSON de cada una.
3. La prueba donde se consume el registro DNS del balanceador de carga y se obtiene una respuesta, se debe ir actualizando la web para obtener la respuesta del objeto JSON de cada una.
4. Se deberá detener la aplicación de la API de una instancia y verificar que la instancia restante responda a las solicitudes.
5. Se deberá volver a iniciar la API detenida y se volverá a realizar la prueba del paso 3 nuevamente.

Fecha de entrega

Agosto xx de 2025

Rúbrica de Calificación

Criterio	Descripción	Puntos Máximos
Creación Inicial de APIs	Se crean dos APIs en lenguajes diferentes y se evidencian los mismos.	0.2
Configuración de APIs	Cada API contiene los dos endpoints solicitados: ***"/check"*** que retorna código 200 y ***"/"*** que retorna un JSON con la estructura requerida	0.3
Despliegue en EC2	Se crean dos instancias EC2 con los nombres **"Instancia-1"** y **"Instancia-2"**..	0.5
Configuración de EC2	Cada API es correctamente desplegada en su instancia correspondiente y se verifica que las APIs estén accesibles y operativas desde su IP pública	0.7
Configuración de ELB (Elastic Load Balancer)	Se crea un balanceador de carga con el nombre correcto: ***"elb-semi1-ht1-#carnet"***, verificando que el ELB distribuye el tráfico correctamente entre las dos instancias.	1
Calidad y duración del video	El video está ordenado de manera adecuada y dentro del marco de 10 minutos como máximo.	0.3
TOTAL		3

Valores

Para el desarrollo de esta hoja de trabajo se espera que el estudiante demuestre responsabilidad e iniciativa académica.

1. **Entregas a tiempo**
    - Cada estudiante debe hacer entrega de su tarea en la fecha y horario establecido sin atrasos, cualquier entrega fuera del mismo tendrá nota de **0**.
2. **Calidad del video**
    - La evidencia visual a través del video es clara y se cumple con lo solicitado dentro del límite de tiempo indicado.